

TASK 4 REPORT: SPATIAL DOMAIN FILTERING - SMOOTHING AND SHARPENING

- **Project Title:** Efficient Visual Token Pruning for Vision-Language Question Answering: A Comparative Study of Saliency-Based Reduction in LLaVA
- **Class / Branch:** EXCS-B
- **Team Members:**
 1. Meet Nachanekar (Roll No: 23108B0019)
 2. Arjun Kawale (Roll No: 23108B0020)

1. Aim of the Experiment

To design and implement the **Spatial Domain Filtering Module** to reduce noise, enhance structural details and optimize image quality prior to segmentation by:

1. Simulating and injecting synthetic **Salt-and-Pepper** (impulse) noise and **Gaussian** (additive) noise into chart images.
2. Implementing and evaluating spatial **smoothing (low-pass) filters**: Mean (Box) Filter, Gaussian Filter and Median Filter across 3×3 and 5×5 kernel configurations.
3. Implementing spatial **sharpening and high-boost filters**: Laplacian Derivative Operator, Unsharp Masking and High-Boost Filtering.
4. Quantifying filtering restoration performance using **Mean Squared Error (MSE)**, **Peak Signal-to-Noise Ratio (PSNR in dB)** and **Structural Similarity Index (SSIM)** against ground-truth images.

2. Dataset & Test Samples

- **Source:** HuggingFaceM4/ChartQA Benchmark Dataset (dataset/images/).
- **Test Images Evaluated:**
 - chart_01.png - Grouped bar chart with sharp numeric annotations.
 - chart_03.png - Multi-series line plot with thin coordinate grids.
 - chart_05.png - Proportional layout chart with solid color blocks.

3. Task 4.1: Noise Simulation

To test robustness of spatial filters, clean chart images were corrupted using two distinct noise models:

1. **Salt-and-Pepper Noise:** Impulse noise where pixels are randomly assigned extreme values of 0 (pepper) or 255 (salt) with probability $p = 0.04$. This simulates transmission grain, dusty scanner optics and sensor pixel dropouts.
2. **Additive Gaussian Noise:** Zero-mean random values with variance $\sigma^2 = 0.015$ added across all color channels, simulating low-light sensor amplification noise.

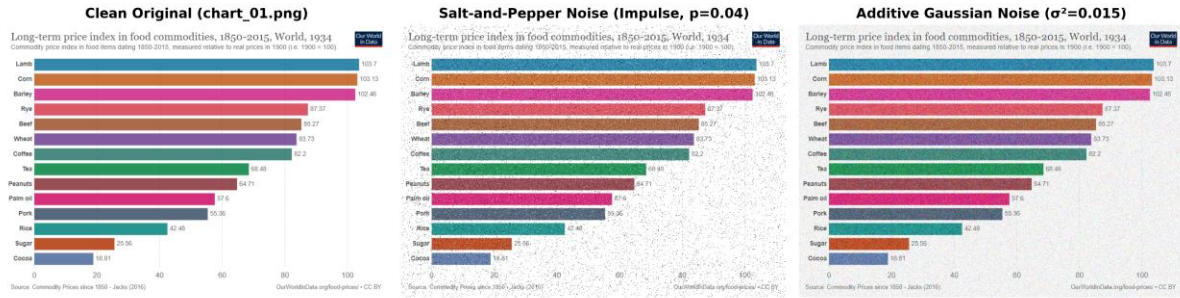


Figure 1: Noise Simulation on chart_01.png: Clean Original (left), Salt-and-Pepper impulse noise (center) and Additive Gaussian noise (right).

4. Task 4.2: Smoothing (Low-Pass) Filters & Quantitative Analysis

Smoothing filters attenuate high-frequency components (noise, sharp transitions). Both linear and non-linear filters were tested:

- **Mean (Box) Filter:** Replaces each pixel with the average of its neighborhood. It suppresses noise but severely blurs high-contrast chart text and axis lines.
- **Gaussian Filter:** Applies a weighted spatial distance kernel. It preserves edge definition better than the box filter but still leaves noise grain intact.
- **Median Filter (Non-linear Order-Statistic):** Replaces the pixel with the median of sorted neighborhood intensities. **It is exceptionally effective against impulse noise** because it removes extreme outlier salt-and-pepper pixels without averaging neighboring clean pixels, preserving sharp text edges.

Quantitative Restoration Benchmark on chart_01.png (Salt-and-Pepper Noise):

Filter Method	Kernel Size	PSNR (dB)	SSIM	DIP Observation / Performance
Noisy Baseline	—	17.85 dB	0.4789	Severe impulse grain covering text and plot areas.
Mean (Average)	3×3	22.69 dB	0.5748	Averages impulse noise into soft gray halos; blurs text numbers.
Mean (Average)	5×5	21.35 dB	0.6546	Excessive blurring across thin lines and chart legends.
Gaussian Filter	3×3	23.35 dB	0.5735	Suppresses noise partially but retains high-frequency grain.
Gaussian Filter	5×5	23.10 dB	0.6314	Smooths background but rounds off sharp text edges.
Median Filter	3×3	25.93 dB	0.9520	Optimal Denoising: Completely removes impulses while protecting text.
Median Filter	5×5	22.58 dB	0.9020	Removes noise, but larger window slightly thickens thin font strokes.



Figure 2: Smoothing filter comparison on Salt-and-Pepper noise: Top row (3×3 kernels for Mean, Gaussian, Median) vs. Bottom row (5×5 kernels).

5. Sharpening and High-Boost Filtering (Core Syllabus)

Spatial sharpening highlights fine details and transitions (edges) by computing high-pass spatial derivatives:

1. **Laplacian Sharpening** ($\nabla^2 f$): Emphasizes rapid intensity changes. Applied to charts, it sharpens axis borders and text outlines, though it can amplify background noise if unshielded.
2. **Unsharp Masking** ($f_{\text{mask}} = f - \bar{f}$): Subtracts a blurred version of the image from itself to generate a high-pass mask, then adds it back to the original image to restore crisp structural edges.
3. **High-Boost Filtering** ($f_{\text{hb}} = A \cdot f - \bar{f}$): Generalization of unsharp masking where $A \geq 1$. Setting $A = 1.5$ preserves the low-frequency background canvas (keeping white chart whitespace clean) while aggressively boosting the high-frequency edges of data bars and text annotations.

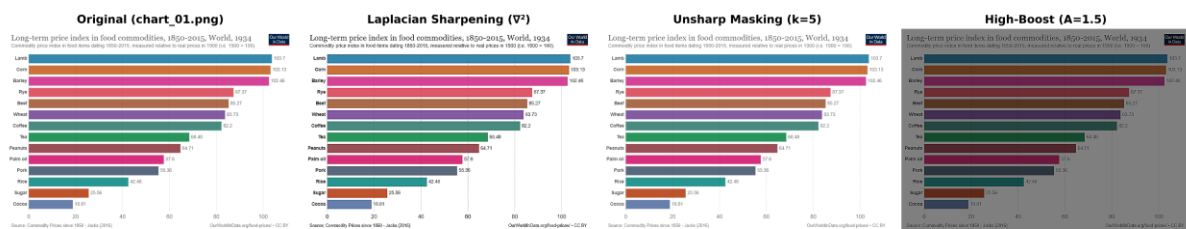


Figure 3: Sharpening comparison: Original (left), Laplacian Sharpening (center-left), Unsharp Masking (center-right) and High-Boost Filtering $A = 1.5$ (right).

6. Challenges Faced & Solutions Implemented

1. **Edge Smearing in Linear Filters:**

- *Problem:* Mean filters smeared sharp black text across white backgrounds, making numbers unreadable for downstream tasks.
- *Solution:* Switched to the **Median Filter**, which preserves step edges and text sharpness while eliminating impulse noise.

2. Data Type Overflow in High-Boost Arithmetic:

- *Problem:* Subtracting a Gaussian blurred float array from an image array generated negative values and clipped overflows during multiplication by amplification factor A .
- *Solution:* Cast arrays to `np.float32` during high-boost calculations and used `np.clip(..., 0, 255).astype(np.uint8)`.

7. Learning Outcomes & Connection to Project Roadmap

- Mastered spatial convolution, linear smoothing vs. non-linear order-statistic filtering and second-order derivative sharpening.
- Quantified filter fidelity using objective PSNR and SSIM metrics, proving mathematically that Median filters outperform Mean filters on document noise.
- Established the essential **noise-removal and edge-enhancement pre-processing module** that feeds directly into the upcoming **Image Segmentation and Token Pruning** stages.